

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME:** Artificial Intelligence

**COURSE CODE:** CSA17

---

### COURSE OUTCOME COVERED

**CO1:** Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

*Assessment Tool Weightage: 50%*

---

**QUESTIONS: 5**

**Duration : 1 Hour**  
**Total Marks:50**

Annexure A

### Analytical Problem Solving

---

#### ***Code of Conduct:***

*I \_\_\_\_\_ HARIHARAN.R/192524003 \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

#### ***Signature of the student:***

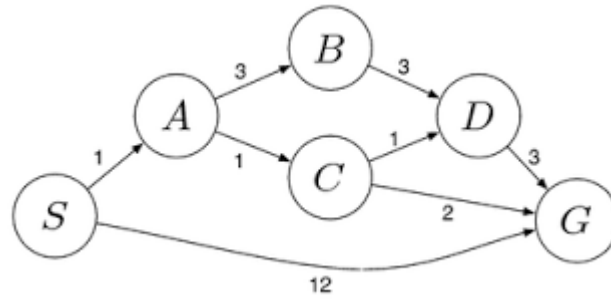
HARIHARAN.R

## Annexure A

### **Analytical Problem Solving**

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
  - i. What are the percept's for this agent?
  - ii. Characterize the operating environment.
  - iii. What are the actions the agent can take?
  - iv. How can one evaluate the performance of the agent?
  - v. What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



**RUBRICS FOR CALCULATION:**

| Criteria                  | Weightage | Level 4<br>(Exemplary)                                                            | Level 3<br>(Proficient)                   | Level 2<br>(Developing)                    | Level 1<br>(Beginning)                        |
|---------------------------|-----------|-----------------------------------------------------------------------------------|-------------------------------------------|--------------------------------------------|-----------------------------------------------|
| Problem Understanding     | 20%       | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps       | Partial understanding; misses key elements | Misinterprets or unable to understand problem |
| Method Selection          | 20%       | Selects most appropriate method/formula with strong justification                 | Selects correct method with minor errors  | Inappropriate or partially correct method  | Unable to select method                       |
| Solution Process          | 25%       | Logical, systematic steps with correct approach throughout                        | Mostly correct steps with minor mistakes  | Disorganized steps; multiple errors        | No clear process                              |
| Accuracy of Calculation   | 20%       | All calculations accurate; correct final answer                                   | Minor calculation errors                  | Frequent errors; incorrect final answer    | No valid calculations                         |
| Interpretation of Results | 15%       | Clearly interprets results with units and engineering relevance                   | Basic interpretation with minor omissions | Limited or unclear interpretation          | No interpretation                             |
| Problem Understanding     | 20%       | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps       | Partial understanding; misses key elements | Misinterprets or unable to understand problem |

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

ANSWER:

### Problem Statement

We have two jugs:

- **4-gallon jug**
- **3-gallon jug**

Neither jug has any measurement markings. We have a **water pump** to fill the jugs, and we can:

1. Fill a jug completely from the pump.
2. Empty a jug onto the ground.
3. Pour water from one jug into the other.
4. There are no other measuring devices.

**Goal:** Obtain exactly 2 gallons of water in the 4-gallon jug.

### Solution Steps

| Step | Action                                                                                | 4-Gallon Jug | 3-Gallon Jug |
|------|---------------------------------------------------------------------------------------|--------------|--------------|
| 1    | Fill the 3-gallon jug completely                                                      | 0            | 3            |
| 2    | Pour the water from the 3-gallon jug into the 4-gallon jug                            | 3            | 0            |
| 3    | Fill the 3-gallon jug completely again                                                | 3            | 3            |
| 4    | Pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full | 4            | 2            |
| 5    | Empty the 4-gallon jug onto the ground                                                | 0            | 2            |
| 6    | Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug              | 2            | 0            |

### Final Result

After completing the above steps:

- 4-gallon jug = exactly 2 gallons
- 3-gallon jug = 0 gallons

Therefore, we have successfully measured exactly 2 gallons of water in the 4-gallon jug.

### State-Space Representation

The solution can also be represented as:

$(0,0) \rightarrow (0,3) \rightarrow (3,0) \rightarrow (3,3) \rightarrow (4,2) \rightarrow (0,2) \rightarrow (2,0)$

Here, each pair (x, y) represents:

### Final Result

After completing the above steps:

- 4-gallon jug = exactly 2 gallons
- 3-gallon jug = 0 gallons

Therefore, we have successfully measured exactly 2 gallons of water in the 4-gallon jug.

### State-Space Representation

The solution can also be represented as:

$(0,0) \rightarrow (0,3) \rightarrow (3,0) \rightarrow (3,3) \rightarrow (4,2) \rightarrow (0,2) \rightarrow (2,0)$

Here, each pair (x, y) represents:

### Conclusion:

**The Water Jug problem demonstrates state-space search in Artificial Intelligence. Each possible configuration of water in the two jugs represents a state, and each operation such as filling, emptying, or transferring water represents an action**

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

1. What are the percept's for this agent?
2. Characterize the operating environment.
3. What are the actions the agent can take?
4. How can one evaluate the performance of the agent?
5. What sort of agent architecture do you think is most suitable for this agent? Why?

### ANSWER:

#### Introduction

A **Mars Rover** is an autonomous robotic vehicle designed to explore the surface of Mars. It collects information about the Martian environment, studies rocks and soil, analyzes samples, takes photographs, and sends scientific data back to Earth. Since communication between Earth and Mars has a significant delay, the rover must be capable of making many decisions autonomously.

#### i. What are the Percepts for this Agent?

**Percepts** are the information that the Mars Rover receives from its environment through its sensors and instruments.

The major percepts of a Mars Rover include:

1. **Images and Videos** – Cameras capture photographs and videos of the Martian surface, rocks, soil, and surroundings.
2. **Distance and Obstacle Information** – Sensors detect rocks, cliffs, holes, and other obstacles in the rover's path.
3. **Terrain Information** – The rover identifies slopes, sand, rocks, and uneven surfaces.
4. **Temperature** – Sensors measure the temperature of the atmosphere and surface.
5. **Atmospheric Conditions** – The rover can collect information about atmospheric pressure, wind, and dust.
6. **Chemical Composition** – Scientific instruments analyze the chemical and mineral composition of rocks and soil.
7. **Sample Information** – Instruments detect characteristics of collected rock and soil samples.
8. **Rover's Internal Status** – The rover monitors battery level, system health, wheel condition, and equipment status.

#### ii. Characterize the Operating Environment

The Mars Rover operates in a challenging environment. Its operating environment can be characterized using the following properties:

| Property      | Characterization     | Explanation                                                     |
|---------------|----------------------|-----------------------------------------------------------------|
| Observability | Partially Observable | The rover cannot see or know everything about its surroundings. |

| Property                        | Characterization    | Explanation                                                                                                 |
|---------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Deterministic/Stochastic</b> | Stochastic          | Weather, dust, terrain, and other conditions can be unpredictable.                                          |
| <b>Episodic/Sequential</b>      | Sequential          | Every action affects future actions and the rover's future state.                                           |
| <b>Static/Dynamic</b>           | Dynamic             | The environment can change due to dust storms, weather, and terrain conditions.                             |
| <b>Discrete/Continuous</b>      | Continuous          | Position, movement, temperature, and other environmental variables change continuously.                     |
| <b>Single/Multi-Agent</b>       | Mostly Single-Agent | The rover mainly operates independently, although it may communicate with orbiters and Earth-based systems. |
| <b>Known/Unknown</b>            | Partially Unknown   | Scientists have limited knowledge of the exact conditions at every location on Mars.                        |

### iii. What are the Actions the Agent Can Take?

The Mars Rover can perform several actions to explore Mars and analyze samples.

#### 1. Movement Actions

- Move forward.
- Move backward.
- Turn left.
- Turn right.
- Change direction.
- Stop.
- Navigate around obstacles.

#### 2. Observation Actions

- Capture images using cameras.
- Scan the surrounding environment.
- Examine rocks and soil.
- Measure atmospheric conditions.

#### 3. Sample Collection Actions

- Select a scientifically interesting rock or soil location.
- Drill into rocks.
- Collect soil or rock samples.
- Store samples for further analysis.

#### 4. Sample Analysis Actions

- Analyze the chemical composition of samples.
- Identify minerals.
- Detect possible signs of past microbial activity.
- Study the physical properties of rocks and soil.

#### 5. Communication Actions

- Send scientific data to Earth.
- Send photographs and analysis results.
- Receive commands from mission control.
- Report its current location and system status.

#### 6. Self-Maintenance Actions

- Monitor battery power.
- Move to a suitable location for solar charging.

- Enter a safe mode when a problem occurs.

#### iv. How Can One Evaluate the Performance of the Agent?

The performance of a Mars Rover can be evaluated using a **performance measure** that determines how successfully it achieves its mission objectives.

The following factors can be used:

##### 1. Scientific Data Collected

The rover should collect useful scientific information about Mars, including data about rocks, soil, and the atmosphere.

##### 2. Quality of Sample Analysis

The rover should accurately analyze collected samples and provide valuable information about their chemical and mineral composition.

##### 3. Exploration Distance

The distance traveled and the number of scientifically important locations explored can be used to measure performance.

##### 4. Successful Sample Collection

The number and quality of rock and soil samples successfully collected can be considered.

#### v. What Sort of Agent Architecture is Most Suitable? Why?

The most suitable architecture for a Mars Rover is a **Hybrid Intelligent Agent Architecture**, combining a **Model-Based, Goal-Based, Utility-Based, and Learning Agent**.

##### 1. Model-Based Agent

The rover needs an internal model of its environment because it cannot directly observe everything around it.

For example, it can maintain information about:

- Its current location.
- Previously explored areas.
- Known obstacles.
- Available energy.
- Terrain conditions.

This helps the rover make decisions even when some information is not directly observable.

## 2. Goal-Based Agent

The rover has specific goals, such as:

- Reach a particular location.
- Explore a new area.
- Collect a rock sample.
- Analyze a soil sample.
- Find evidence of past water or possible life.

The rover selects actions that help it achieve these goals.

## Conclusion

The Mars Rover is an excellent example of an **intelligent agent** operating in a complex real-world environment. It uses sensors to perceive Mars, processes the collected information, makes decisions, and performs actions such as moving, collecting samples, analyzing rocks and soil, and communicating with Earth. Since Mars is partially observable, uncertain, dynamic, and difficult to access, a **hybrid agent architecture** combining model-based, goal-based, utility-based, and learning capabilities is the most appropriate. This architecture allows the rover to operate autonomously and efficiently while maximizing scientific discoveries and ensuring its safety.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

ANSWER:

## 1. Introduction

The **8-Queens Problem** is a classic problem in **Artificial Intelligence** and **state-space search**. The objective is to place **8 queens on an  $8 \times 8$  chessboard** such that **no two queens attack each other**.

A queen in chess can attack another queen if they are in the same:

- **Row**
- **Column**
- **Diagonal**

Therefore, a valid solution must satisfy all three conditions.

## Objective



Place 8 queens on an  $8 \times 8$  chessboard so that no two queens share the same row, column, or diagonal.

One possible solution is represented by the following column positions for each row:

[1, 5, 8, 6, 3, 7, 2, 4]

This means:

- Queen 1  $\rightarrow$  Row 1, Column 1
- Queen 2  $\rightarrow$  Row 2, Column 5
- Queen 3  $\rightarrow$  Row 3, Column 8
- Queen 4  $\rightarrow$  Row 4, Column 6
- Queen 5  $\rightarrow$  Row 5, Column 3
- Queen 6  $\rightarrow$  Row 6, Column 7
- Queen 7  $\rightarrow$  Row 7, Column 2
- Queen 8  $\rightarrow$  Row 8, Column 4

A visual representation is:

```
1 2 3 4 5 6 7 8
+-----+
1 | Q . . . . . |
2 | . . . . Q . . |
3 | . . . . . Q |
4 | . . . . . Q . |
5 | . . Q . . . . |
6 | . . . . . Q . |
7 | . Q . . . . . |
8 | . . . Q . . . |
+-----+
```

## 2. Problem Formulation

The 8-Queens problem can be formulated as a **state-space search problem** using the following components.

### A. Initial State

The initial state is an **empty  $8 \times 8$  chessboard** with no queens placed.

.....

.....

.....

.....

.....

.....

.....

.....

Initial State = **No queens placed**

---

## **B. State**

A state represents a particular configuration of queens on the chessboard.

For example:

**[1, 5, 8]**

means that queens have been placed in the first three rows at columns 1, 5, and 8.

Q.....

....Q...

.....Q

.....

.....

.....

.....

.....

The state can be represented as:

**State = Positions of queens already placed on the board**

### C. Actions / Operators

An action is the process of placing a queen in an empty square.

The main action is:

**Place a queen in an empty square that does not conflict with any previously placed queen.**

A queen can be placed only if:

1. No queen exists in the same column.
2. No queen exists on the same main diagonal.
3. No queen exists on the same secondary diagonal.

For example, if a queen is placed at position **(row, column) = (2, 5)**, another queen cannot be placed at:

- Any square in row 2.
- Any square in column 5.
- Any square on either diagonal passing through (2,5).

### D. Transition Model

The transition model describes how the state changes after an action.

For example:

```
Empty Board
  ↓
Place Queen 1
  ↓
Place Queen 2
  ↓
Place Queen 3
  ↓
...
  ↓
Place Queen 8
  ↓
Goal State
```

A transition is valid only when the newly placed queen does not attack any existing queen.

### E. Goal Test

The goal test checks whether:

**8 queens have been placed and no two queens attack each other.**

Mathematically, for any two queens at positions  $(r_1, c_1)$  and  $(r_2, c_2)$ :

- They must not be in the same row:  
 $r_1 \neq r_2$
- They must not be in the same column:  
 $c_1 \neq c_2$
- They must not be on the same diagonal:  
 $|r_1 - r_2| \neq |c_1 - c_2|$

If all conditions are satisfied, the state is a **solution**.

## F. Path Cost

The path cost can be defined as the number of queen placements required to reach the goal.

Since exactly 8 queens must be placed:

Path Cost = 8

However, in practical search algorithms, the cost may also represent the number of states explored or computational effort required to find a solution.

## 3. State-Space Search

The 8-Queens problem can be solved by exploring the state space.

The search starts from the empty board and progressively places queens.

Empty Board

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

-----

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

Q at      Q at      Q at

Column 1    Column 2    Column 3

Valid States

Place Next Queen

|

...

|

**8 Queens Placed**

|

**Check Goal Test**

/ \

**Yes      No**

|      |

**Solution      Backtrack**

**The search algorithm explores possible configurations until it finds a valid solution.**

4. A customer wants to travel from one location to another using the OLA Cab booking mobile application. The customer can select any of the cab types as mini, micro, sedan, shared, prime, and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

ANSWER:

## 1. Introduction

A customer wants to travel from a source location to a destination location using an online cab booking application such as OLA. The customer can choose different cab types such as Mini, Micro, Sedan, Prime, Shared, etc., depending on their comfort, budget, travel time, and other requirements.

This problem can be solved using a Utility-Based Agent.

## 2. Type of Problem-Solving Agent

### Utility-Based Agent

A Utility-Based Agent is the most suitable agent for this problem.

The agent evaluates different cab options based on their utility or overall benefit to the customer. It considers factors such as:

- Cab type
- Fare/cost
- Distance
- Estimated travel time
- Customer comfort
- Availability of the cab
- Number of passengers
- Customer preference

The agent then selects the cab that provides the maximum utility according to the customer's requirements

| Cab Type | Cost      | Comfort   | Availability |
|----------|-----------|-----------|--------------|
| Micro    | Low       | Medium    | Available    |
| Mini     | Medium    | Medium    | Available    |
| Sedan    | High      | High      | Available    |
| Prime    | Very High | Very High | Available    |
| Shared   | Very Low  | Low       | Available    |

**. Pseudocode:**

**ALGORITHM OLA\_CAB\_BOOKING\_AGENT**

**BEGIN**

**DISPLAY "Enter Source Location"**

**INPUT source**

**DISPLAY "Enter Destination Location"**

**INPUT destination**

**DISPLAY "Enter Number of Passengers"**

**INPUT passengers**

**DISPLAY "Enter Customer Preference"**

**DISPLAY "1. Low Cost"**

**DISPLAY "2. High Comfort"**

**DISPLAY "3. Minimum Travel Time"**

**DISPLAY "4. Balanced Option"**

**INPUT preference**

**IF source = destination THEN**

**DISPLAY "Source and destination cannot be the same"**

**STOP**

**END IF**

**SEARCH for available cabs from source to destination**

**IF no cab is available THEN**

**DISPLAY "No cabs available"**

**STOP**

**END IF**

**FOR each available cab type DO**

**GET cab\_type**

**GET fare**

**GET estimated\_time**

**GET comfort\_level**

**GET availability**

**IF passengers > cab\_capacity THEN**

**REMOVE cab\_type from available options**

**END IF**

**END FOR**

**IF preference = "Low Cost" THEN**

**SELECT cab with minimum fare**



**ELSE IF preference = "High Comfort" THEN**

**SELECT cab with maximum comfort**

**ELSE IF preference = "Minimum Travel Time" THEN**

**SELECT cab with minimum estimated travel time**

**ELSE**

**CALCULATE utility for each cab**

**utility =**

**weight\_cost × cost\_score**

**+ weight\_comfort × comfort\_score**

**+ weight\_time × time\_score**

**SELECT cab with maximum utility**

**END IF**

**DISPLAY selected cab type**

**DISPLAY estimated fare**

**DISPLAY estimated travel time**

**ASK customer:**

**"Do you want to book this cab?"**

**INPUT choice**

**IF choice = YES THEN**

**BOOK selected cab**

**DISPLAY "Cab booked successfully"**

**TRACK cab location**

**WHEN cab arrives:**

**START JOURNEY**

**WHEN destination is reached:**

**DISPLAY "Journey completed successfully"**

**ELSE**

**DISPLAY "Booking cancelled"**

**END IF**

**END**

#### **4. Flow of the Agent**

**START**

|

↓

**Enter Source Location**

|

↓

**Enter Destination Location**



**Enter Passenger Details**



**Select User Preference**



**Search Available Cabs**



**Are Cabs Available?**

/ \

**NO**

**YES**



**Display No      Compare Cab**

**Cab      Types, Cost,**

**Available      Time & Comfort**



**Calculate Utility**



**Select Best Cab**

|

↓

**Display Cab Details**

|

↓

**Customer Confirms?**

/ \

**NO      YES**

|

|

↓

↓

**Cancel Booking   Book Cab**

|

↓

**Track Cab**

|

↓

**Start Trip**

|

↓

**Reach Destination**

|

↓

**END**

## **Conclusion**

**The OLA Cab Booking problem can be modeled as a Utility-Based Problem-Solving Agent. The agent receives the source, destination, passenger details, and customer preferences as input. It searches for available cab types, evaluates them based on cost, comfort, travel time, and availability, and recommends the most suitable cab. After customer confirmation, it books and tracks the cab until the customer reaches the destination. This approach provides a personalized and efficient cab-booking experience.**

5.A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:  
ANSWER:

## **Introduction**

**A logistics company wants to transport packages from a starting warehouse S to a goal warehouse G at the minimum total cost. Each route between warehouses has an associated cost representing factors such as fuel consumption and travel time.**

### **1. Problem Formulation**

#### **2. Initial State**

**3. The initial state is the starting warehouse S.**

#### **4. Goal State**

**5. The goal state is the destination warehouse G.**

#### **6. Actions**

**7. From the current warehouse, the agent can travel to any directly connected neighboring warehouse.**

#### **8. Path Cost**

**9. The path cost is the sum of all route costs from S to G.**

**10.For example, if:**

**11. $S \rightarrow A \rightarrow B \rightarrow G$**

**12.and the route costs are:**

**13. $S \rightarrow A = 2$**

**14. $A \rightarrow B = 4$**

**15. $B \rightarrow G = 3$**

**16.Then:**

**17.Total Cost =  $2 + 4 + 3 = 9$**

**18.Goal**

**19.Find the path from S to G with the minimum total cost.**

## 2. Uniform Cost Search (UCS)

Uniform Cost Search is an uninformed search algorithm that always expands the node with the lowest cumulative path cost from the starting node.

The cumulative path cost is represented by:

$g(n)$  = total cost from S to node n

The main idea is:

Always expand the least-cost node first.

UCS uses a Priority Queue where nodes are ordered according to their cumulative path cost.

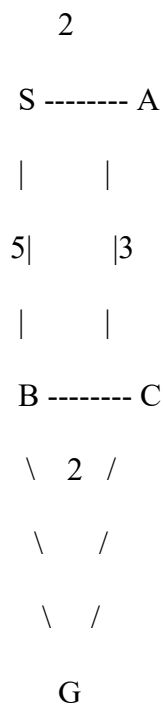
## 3. UCS Algorithm

The general process is:

1. Start at node S with cost 0.
2. Add S to the priority queue.
3. Select the node with the smallest cumulative cost.
4. If the selected node is G, stop. The current path is the optimal path.
5. Otherwise, expand the node.
6. Calculate the cumulative cost for each successor.
7. Add the successors to the priority queue.
8. Repeat until G is selected for expansion.

## 4. UCS Example

Consider the following example weighted graph:



## 5. UCS Search Process

| 6. Step | 7. Expanded Node | 8. Frontier / Priority Queue | 9. Cost               |
|---------|------------------|------------------------------|-----------------------|
| 10. 1   | 11. S            | 12. A, B                     | 13. A=2, B=5          |
| 14. 2   | 15. A            | 16. B, C                     | 17. B=5, C=5          |
| 18. 3   | 19. B or C       | 20. Remaining nodes          | 21. Lowest cost first |
| 22. 4   | 23. C            | 24. G                        | 25. G=9               |
| 26. 5   | 27. G            | 28. —                        | 29. <b>9</b>          |

## Conclusion

The warehouse delivery problem can be modeled as a **weighted graph**, where warehouses are represented as **nodes** and transportation routes are represented as **edges with associated costs**. The **Uniform Cost Search (UCS)** algorithm starts from warehouse **S** and always expands the warehouse with the **lowest cumulative transportation cost**. When the goal warehouse **G** is reached, UCS provides the optimal least-cost route, provided that all route costs are non-negative.

For the example graph used above, the optimal route is:

**S → A → C → G**